

Session 1

Introduction to AI Development Workflows



Four parts, thirty minutes.

01	Where AI fits in the workflow	5 min
02	Models, coding agents and MCP	8 min
03	Prompting, and how to check the answer	12 min
04	Live demo & questions	5 min

AI shortens every stage. It removes none of them.

Stage 1

Ideate

Describe the device in plain language. AI turns the description into a spec you can argue with.

Stage 2

Plug

Snap on the sensors. The board reports what is connected, so the AI starts with facts.

Stage 3

Build

Prompt for the firmware. Workspace AI compiles it in our own toolchain, so it either builds or it does not.

Stage 4

Debug

Read the logs, describe the symptom, ask for causes to test. Do not ask for a rewrite.

Stage 5

Hand Off

Pin maps, README, known issues. The cheapest and most reliable use of AI in the whole list.

Different models for different jobs.

DAILY TASKS

Google Gemini

Explains and drafts

Datasheets, error messages, trade-offs, first drafts. The one to keep open all day.

Sees: only what you paste

CODING AGENTS

Claude Code Codex

Works in your project

Run in the terminal, read the whole repository, edit files and run the build themselves.

Sees: every file you allow

VIDEO

Seedance

Generates footage

Short clips from a text or image prompt, for demo reels and project write-ups.

Sees: your prompt only

HARDWARE CONTEXT

INCIPE Workspace

Knows your board

Our software speaks the board's communication protocol, so the AI already knows what is connected and what it is doing.

Sees: the board and the project

Only the Workspace sees the hardware. The other three answer from what you tell them.

Role, Goal, Context. Then the details.

Role	Who the AI should act as, and who it is writing for.
Goal	One outcome, stated as a verb. Not two.
Context	The board, the chip, the language, what you already tried.
Constraints	Limits it must respect: memory, no libraries, no rewrite.
Format	Code only, a table, three bullet points.
Check	How you will know the answer is right.

ALL SIX, IN ONE PROMPT

```
You are an embedded firmware engineer writing for a beginner. Write one C function that reads temperature and humidity from the DHT module on the INCIPE — 1 board.

Use only the standard board library and no dynamic allocation. Return the code and a one-line comment per step, nothing else. Then tell me what a valid reading looks like so I can check it on the serial monitor.
```

Three sentences. Every one of the six parts is in there.

Name the board, the pin and the limit.

TOO VAGUE

Write code to read a temperature sensor.

You get code for some sensor on some chip. It may not compile, and you cannot tell whether it is wrong or just different.

SPECIFIC ENOUGH TO USE

In C for the INCIPE — 1 board, write a function that reads temperature and humidity from the DHT module. No dynamic allocation. Return a struct and handle a read timeout. Code only.

You get code you can compile, flash and test in one pass.

Ask for causes to test, not for a rewrite.

TOO VAGUE

My sensor code does not work. Fix it.

You get a guess, usually a full rewrite that quietly replaces your working code and hides the real fault.

SPECIFIC ENOUGH TO USE

This read returns -999 on every third call. Here is the function and the serial log. I have already checked the wiring and the module seating. List the three most likely causes, most likely first, and how to test each. Do not rewrite the function.

You get causes you can test one at a time, and you keep your own code.

Name the reader, the sections and the length.

TOO VAGUE

Document this project.

You get filler at the wrong length for the wrong reader, and some of it will be invented.

SPECIFIC ENOUGH TO USE

Write a README for this project aimed at a classmate who has never seen it. Sections: what it does, how to flash it, the pin map as a table, known issues. Under 400 words. Use only facts from the project files, and list anything you could not find.

You get a draft that needs editing rather than rewriting.

Give it a marker, then watch for the marker.

SET THE MARKER

Starting from this prompt, answer me in this format, with a divider between each section.

1. Say the word ON.
2. Answer the question.
3. List what you assumed.

READ THE SIGNAL

The word ON costs nothing and proves the model is still reading your instructions. When ON disappears, or the dividers drift, your context is gone — anything after that point is worth less than it looks.

TWO MORE CHECKS

Ask it to cite. Which line of the file did you take that from?

Ask it to separate. Which part of this did you read, and which part did you infer?

When the marker stops appearing, start a new chat and paste the prompt again. Do not argue with a model that has lost the thread.

Make it ask you the questions first.

BEFORE IT WRITES ANYTHING

Before you write any code, grill me. Ask up to five questions about anything you need to know about the board, the sensors or what I am trying to build. Do not guess, and do not start until I have answered.

Cap the number, or it will keep asking.

WHY IT WORKS

A model that cannot ask will assume. Its questions tell you exactly which parts of your own brief are missing — usually the pin, the units, or the update rate.

THE OTHER DIRECTION

Grill me on this design. What have I not thought about that will break during the demo?

Useful once the thing works and you are about to show it to somebody.

A prompt that worked is worth keeping.

BLANK OUT THE SPECIFICS

In C for the **[BOARD]**, write a function that reads **[VALUE]** from the **[MODULE]**. No dynamic allocation. Return a struct and handle a read timeout. Code only.

The same skeleton covers the next sensor, and the one after that.

WHERE TO KEEP THEM

One prompts file per project, next to the code. Three or four that you trust beat a folder of fifty you have never re-read.

WORTH RECYCLING

The driver prompt. The debug prompt. The README prompt. The grill-me prompt.

Note what you changed and why, in one line, at the top of the file.

MCP lets the agent use your other tools.

WHAT IT IS

Model Context Protocol is a common plug for connecting an AI agent to a program that already exists. The agent stops describing what to do and does it: opens the file, changes the object, runs the build.

Without MCP the agent can only give you the steps.
With MCP it takes them, and your job moves from typing to reviewing.

THREE PIECES

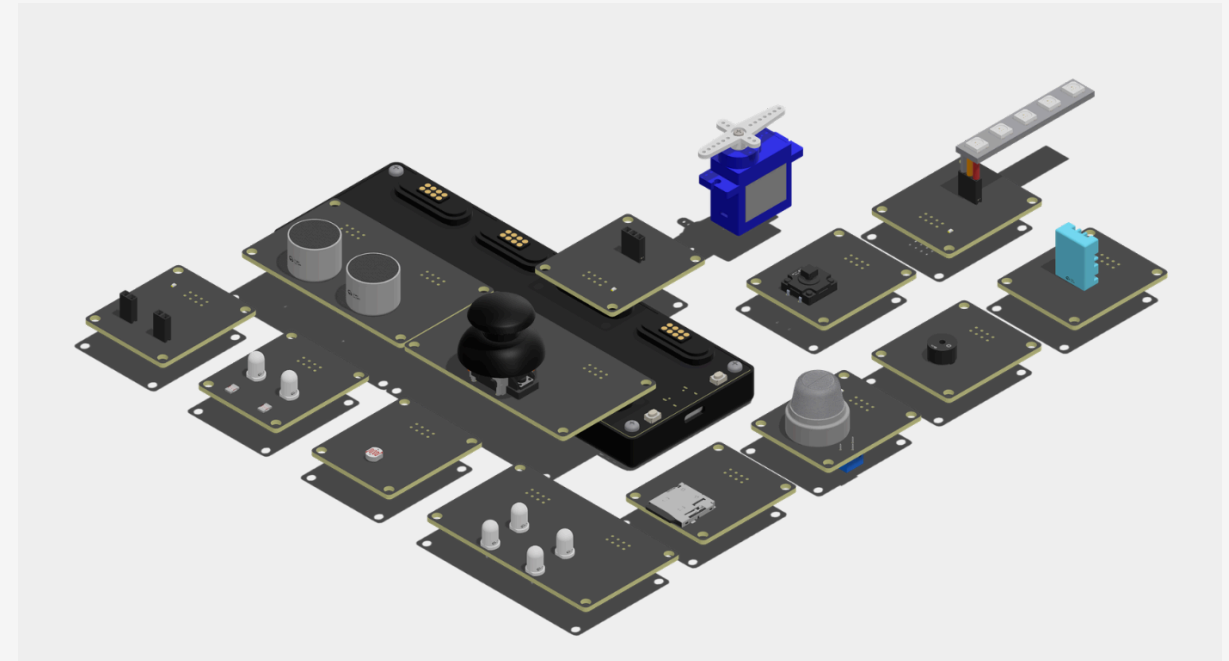
The agent	Claude Code, in your terminal.
The server	A small program that exposes one tool's commands as things the agent may call.
The tool	Blender, a database, a browser, a version control host, our Workspace.

Retexturing the board without touching Blender.

ONE PROMPT, IN CLAUDE CODE

Using the Blender MCP, open board.blend, select the top shell of the INCIPE — 1 and give it a matte black anodised finish. Keep the silkscreen legible. Render one frame from the current camera and save it next to the file.

- 1 Claude Code calls the Blender MCP server.
- 2 Blender assigns the material and sets the roughness.
- 3 The render comes back as an image you can look at.
- 4 You judge it and prompt the next change.



The board asset the agent edits.

The checks still apply. Ask which object it changed and which property it set, and look at the render before accepting it.

Three things to take away.

- 1 AI shortens the drafting inside every stage. It does not remove a stage, and it does not remove you.
- 2 Pick the agent by how much of your project the answer depends on.
- 3 Role, Goal, Context — then set a marker so you can tell when the model has lost them.

SESSION 2

AI-Powered Firmware Development

Using the INCIPE Workspace AI agent for firmware compilation. AI-assisted code review and optimization. Understanding the limits of AI: when to trust it, and when to write your own code.

Practice. Use AI to generate a function that reads from the Temperature & Humidity sensor, then manually optimize the memory usage.